INTERNSHIP PROJECT REPORT

Aureon

A Requirement Intelligence and Project Scoping Platform

Project Version 0.1.0

Student Name	[Enter your name]
Register Number	[Enter your register number]
Institution	[Enter your college or university]
Organisation	[Enter organisation]
Internship Dates	[Enter start and end dates]
Project Guide	[Enter guide or mentor name]

October 2026

Project Overview

Aureon is a web-based requirement intelligence and project scoping platform. I built it to address a practical problem that appears before software development begins: client requirements are often incomplete, mixed with assumptions, or written without enough technical detail for estimation. The application takes a raw idea or requirements document and turns it into a structured scope that can be reviewed by both a client and an engineering team.

The system does more than summarize text. It checks nine requirement areas, identifies what is present and what is missing, searches public GitHub repositories for useful implementation patterns, computes a confidence score using server-side rules, and asks focused clarification questions when the scope is not ready. When enough information is available, it generates two editable outputs: a client-facing scope and an engineer-facing implementation plan.

My main conclusion from the project is that large language models are useful for interpretation and drafting, but they should not control every decision. In Aureon, the model extracts and explains information, while confidence, blockers, effort, risk rules, access control, rate limits, versioning, and validation remain deterministic application logic. This division made the application easier to test and reduced the effect of inconsistent model responses.

Objectives

- Convert vague or unstructured client input into an organized software scope.
- Check the project goal, users, features, stack, integrations, non-functional requirements, budget, timeline, and success criteria.
- Ask only the clarification questions that can materially improve the scope.
- Use public GitHub repositories as supporting evidence without allowing repository content to control the system.
- Produce separate client and engineering views with editable sections and export options.
- Preserve versions, approvals, share links, actual effort, and calibration data for future projects.
- Support local development with minimal setup while retaining a clear path to production deployment.

Scope of the Implemented System

The implemented application covers the full scoping workflow from input to versioned output. It includes text and document import, requirement extraction, GitHub enrichment, clarification rounds, deterministic scoring, final document generation, project history, sharing, approval, exports, and estimate calibration. Live collaboration, billing, vector search, and two-way issue-tracker synchronization are outside the current version.

Area	Current state	Comment
Requirement extraction	Implemented	Nine parameter states with evidence and gaps
Clarification rounds	Implemented	MCQ, blocker question, and added description paths
GitHub enrichment	Implemented	Lexical retrieval, README analysis, and folder mining
Client and engineer views	Implemented	Editable, versioned, and exportable
Accounts and workspaces	Implemented	Auth.js, Google login, and email magic link support
Calibration	Implemented	Uses completed-project actuals after five eligible records
Vector embeddings	Not included	The current retrieval approach is lexical

Area	Current state	Comment
Live co-editing	Not included	Sharing and approval are version based
Self-service billing	Not included	No payment provider is required by Aureon itself

Problem Analysis and Design Approach

The central design problem was not simply generating a document. A useful scope must distinguish confirmed information from assumptions, stop when essential information is missing, and explain how its conclusions were reached. A plain chat interface could produce fluent text, but it would be difficult to measure, compare, or audit. I therefore represented the scope as validated structured data before generating any long-form output.

Requirement Model

Each round evaluates nine fixed parameters. Every parameter is marked as PRESENT, PARTIAL, or MISSING and includes evidence or a gap. The fixed model makes different projects comparable and supports deterministic scoring.

ID	Parameter	Purpose
P-01	Project goal	Defines the problem and intended outcome
P-02	Target users	Identifies roles and affected user groups
P-03	Core features	Defines the minimum functional scope
P-04	Technical constraints	Captures stack, platform, and architecture choices
P-05	Integrations	Records external systems and providers
P-06	Non-functional requirements	Covers security, performance, accessibility, and reliability
P-07	Budget	Provides a commercial delivery constraint
P-08	Timeline	Sets the expected delivery window
P-09	Success criteria	Defines measurable acceptance conditions

Why the Score Is Deterministic

The model reports clarity and structured evidence, but it does not decide its own confidence score. The server calculates confidence from coverage, extraction quality, consistency, and a small GitHub evidence bonus. Caps are applied when the project goal, users, core features, or technical constraints are missing. This prevents a fluent response from being mistaken for a complete scope.

The same principle is used for blockers, risk classification, module confidence, effort points, velocity, sprint estimates, and exit rules. The language model writes explanations around these results, while application code remains the source of truth.

User Workflow

1. The user pastes requirements or imports a TXT, Markdown, PDF, or DOCX file.

2. The server extracts structured requirements and detects the project domain.
3. The retrieval stage searches related GitHub repositories and mines useful module patterns.
4. The server calculates confidence and determines whether the scope is ready, blocked, or needs clarification.
5. The user answers focused questions or chooses to generate a lower-confidence draft.
6. The application generates client and engineering documents in parallel.
7. The user edits, exports, shares, approves, versions, and later records actual effort.

System Architecture

Aureon is implemented as a full-stack Next.js application. The browser handles the interactive workflow, while route handlers validate requests, enforce ownership, and queue background work. Postgres stores durable application data. Redis is used for short-lived coordination data such as locks, rate limits, caches, and live progress. In local development, PGlite and in-memory fallbacks allow the project to run without hosted infrastructure.

Layer	Main components	Responsibility
Presentation	Next.js pages, React 19, Tailwind CSS, GSAP	Chat, pipeline status, editing, project history, sharing, and analytics
Application API	App Router route handlers and Zod	Validation, authorization, session operations, imports, exports, and job submission
Pipeline	Orchestrator, extraction, retrieval, scoring, clarification, finalization	Converts input into structured and long-form outputs
Model routing	Gemini and OpenRouter adapters	Routes main and fast tasks and supports provider fallback
Background work	QStash in production, Next.js after locally	Runs model work without holding an HTTP request open
Data	Postgres, Drizzle ORM, PGlite	Stores users, workspaces, projects, versions, shares, decisions, and actuals
Coordination	Upstash Redis or in-memory fallback	Locks, rate limits, caches, budgets, and session events
External retrieval	GitHub REST API and Octokit	Searches repositories, reads READMEs, and inspects source folders

Request and Job Flow

Long model calls are not executed inside the user-facing HTTP request. The run, answer, and finalize endpoints validate the request, acquire a per-session lock, persist the next state, and return HTTP 202. A background job continues the work. The browser receives stage changes through Server-Sent Events and falls back to polling when the stream is unavailable.

In production, QStash delivers signed job requests to dedicated job routes. Locally, Next.js after() executes the job in the same process. Finalization is split into client and engineering jobs so one completed document can be displayed even if the other is still running. Session locks and idempotent job claims prevent duplicate button clicks or repeated deliveries from starting duplicate model work.

Storage Model

Durable records are stored in Postgres through Drizzle ORM. This includes accounts, workspaces, project sessions, extracted rounds, final output, immutable versions, approvals, share links, and actual effort. Redis does not act as the project database; it only stores data that may safely expire. Local development automatically uses a file-backed PGlite database when DATABASE_URL is absent.

Requirement Intelligence Pipeline

Extraction

The first model call converts the user brief into a validated extraction object. Prompts require evidence for each parameter, domain information, modules, assumptions, open questions, and blocker status. Zod validates the tool output. If the response is truncated or invalid, the structured-call wrapper retries once with a repair instruction. Untrusted input is bounded before it enters the prompt.

GitHub Retrieval and Enrichment

The retrieval stage builds several search variants from the domain, important features, and stack hints. Results from the variants are combined, ranked using lexical signals, and narrowed before the system fetches README and folder information. BM25 is used to select the candidates that deserve deeper inspection. This approach provides useful references without requiring an embedding model or vector database.

The original implementation deep-scored six READMEs per round. During live testing this created unnecessary latency and consumed the limited free-model quota quickly. I reduced the deep analysis to the two strongest candidates while keeping real folder evidence from the remaining repositories. One normal first round therefore uses about four model calls instead of up to eight. If GitHub's anonymous rate limit is exhausted, ranked search links are still returned with a clear search-only label.

Scoring and Clarification

After retrieval, the application merges supported module signals, calculates module confidence, and evaluates whether the project can move forward. A blocker is raised when the goal or core functionality is too unclear. Otherwise the server either marks the project ready or asks targeted multiple-choice questions. Clarification rounds are capped to avoid an endless conversation.

Final Documents

When the scope is ready, two background jobs generate different views of the same structured project. The client view uses accessible language and focuses on outcomes, modules, exclusions, and open items. The engineering view adds architecture, technical constraints, implementation modules, dependencies, risks, story points, sprint planning, and estimation details. The user can edit every section, and each saved change creates a version that can be compared with earlier versions.

Model Routing

The provider layer supports separate main and fast routes. Extraction, clarification, and final documents use the main route. GitHub README interpretation uses the fast route. Gemini and OpenRouter implement the same internal client interface, so pipeline code does not depend on provider-specific response formats. Tier-specific OpenRouter keys allow heavy and lightweight tasks to use different quotas, and an optional fallback provider can be tried when the selected provider fails.

Frontend and Interaction Design

The interface follows a dark blueprint visual style. The main screen keeps the requirements conversation in the larger content area and uses a compact pipeline panel on the left. The user always sees which stage is running, what has completed, and whether attention is required.

Progress Experience

An earlier version used a tall animated roadmap that occupied too much horizontal and vertical space. It looked decorative but made the actual brief difficult to read, and a long-running call appeared frozen. I replaced it with five compact stage cards, a progress bar, an elapsed timer, and an active-stage message. After 35 seconds the interface explains that free providers may be slow; after 90 seconds it reports that automatic retries are still running. This change does not hide latency, but it makes the system state understandable.

Animation Decisions

- Stage cards enter with a short staggered motion instead of a long page animation.
- Only the current stage receives a breathing highlight and pulse, which draws attention without making the whole page move.
- The progress bar changes smoothly when a server stage update arrives.
- Completed, pending, running, and error states use different icons and color treatments.
- The interface respects the reduced-motion operating-system preference.
- The desktop pipeline column is constrained to 320 to 360 pixels so the requirements remain readable.

Editing and Output Experience

Generated sections are editable rather than presented as a fixed answer. The application includes an unsaved-changes guard, local autosave behavior, section copy buttons, version comparison, sharing, approval or change requests, and exports to Markdown, PDF, Word, Jira CSV, and Linear CSV. Imported files are converted to text and placed back into the input box for review before a session is created.

Responsive Behavior

Desktop layouts use a fixed-width pipeline column and a flexible content area. On smaller screens the pipeline becomes a compact panel above the conversation. Inputs, question cards, dialogs, and export controls are designed for keyboard and touch use, although a complete accessibility audit and physical-device test pass are still required before production sign-off.

Security Privacy and Reliability

A project brief can contain confidential commercial information, so security controls are part of the application architecture rather than a final add-on. Server-side environment validation prevents unsupported production configurations. Secrets remain in server variables and are never placed in NEXT_PUBLIC settings.

Implemented Controls

Control	Implementation
Ownership	Anonymous ownership cookies and signed-in workspace access checks
Validation	Zod schemas for API input, model output, stored data, and exports
Concurrency	Per-session locks and idempotent job claims
Rate limiting	Separate limits for general, model, import, share, and auth routes
Content Security Policy	Per-request nonce with additional response security headers
Prompt safety	Input length limits, delimiters, output validation, and untrusted README filtering
Budget control	Daily logical model-call counter with a server-side limit
Retention	Raw input is cleared after the configured inactive period; deletion is a hard delete
Share safety	Version-specific links, expiry and revocation support, and read-only access
Production guard	Refuses unsafe storage or free-model production setups unless explicitly allowed

Privacy Behavior

Imported files are read in memory for one request and are not stored as files. The extracted text is shown to the user for correction before submission. Raw project input is retained separately from generated documents and can be cleared according to workspace retention settings. The privacy page lists the processors that may receive data, including the chosen model provider, GitHub, hosting, storage, queue, and email services.

Failure Handling

Model transport errors are converted into stored session errors with a retryable flag. Completed answers and enriched input are preserved so a retry does not start from zero. GitHub enrichment is non-fatal: a GitHub problem reduces the supporting evidence but does not prevent scope generation. Server-Sent Events fall back to polling, and stale jobs can be retried after the lock expires.

Testing and Results

I used unit, integration, browser, evaluation, and live smoke-test layers. Automated tests replace provider credentials with dummy values and block non-local network access, so the normal test suite cannot accidentally spend quota or send project text to an external service.

Test activity	Result	Evidence
TypeScript type check	Passed	tsc --noEmit completed without errors
ESLint	Passed	Repository lint completed without errors
Vitest suite	Passed	70 files and 1,035 tests passed on 23 October 2026
Focused UI and routing tests	Passed	Pipeline, polling, GSAP, provider-routing, and adapter checks
Live extraction	Passed	OpenRouter returned a valid structured extraction
Live full scope round	Passed	17.4 seconds, ready state, confidence 91
Live finalization	Passed	Client and engineering outputs completed in 72.6 seconds
End-to-end smoke checks	Passed	10 of 10 checks passed; API responses stayed below two seconds
Deep GitHub access	Partially verified	Search works; authenticated token is needed after anonymous quota is exhausted
Production deployment	Pending	Requires hosted Postgres, Redis, durable queue, auth, and production provider settings

What the Automated Tests Cover

- Confidence calculation, blocker gates, effort estimation, risk rules, and module confidence.
- Provider routing, structured output repair, budgets, timeouts, and OpenRouter response conversion.
- GitHub query construction, BM25 ranking, request budgets, cache behavior, and untrusted README filtering.
- Session creation, ownership, authorization, rate limits, locks, background jobs, recovery, and retention.
- Imports, exports, sharing, approvals, version history, diffing, actuals, and calibration.
- Chat progress states, error messages, low-confidence generation, editing, autosave, and output controls.

Performance Observation

The first live round completed well within the 45-second smoke-test target. Finalization was slower because the engineering document is much larger and the free OpenRouter endpoint was busy. The client document completed first, while the engineering job continued independently. The results show that the background-job design works, but a stable production model tier is required for predictable latency.

Problems Encountered and Resolutions

Problem	Cause	Resolution
AI service returned HTTP 500	Provider denial, unavailable free model, quota pressure, or malformed tool output	Added provider routing, tier-specific keys, fallback handling, strict tool validation, body timeout handling, and retries
Pipeline appeared frozen	A long free-provider call had no useful visible timing information	Added stage-specific status, elapsed time, saved-progress messages, and clearer retry states
One cycle used too many calls	Six README analyses were added to extraction and clarification calls	Reduced deep README analysis to two while preserving folder-based evidence
GitHub enrichment returned no matches	Anonymous GitHub request limit produced 403 responses for README and tree fetches	Kept ranked search-only references and documented GITHUB_TOKEN for full enrichment
Gemini key did not work	The related Google project was denied API access	Moved local testing to OpenRouter without changing pipeline code
Free model returned protocol variations	Some models returned JSON as content instead of a formal tool call	Added conservative JSON recovery followed by the same schema validation
Long requests risked duplicate work	Retries, double clicks, and at-least-once queue delivery can repeat a job	Used session locks, job claims, stored stages, and idempotent decisions

Key Learning

The most important lesson was that model quality is only one part of an AI-enabled application. Reliability depended more on timeouts, structured validation, state persistence, clear failure messages, quotas, provider behavior, and deterministic business rules. I also learned that an animated loading screen is not enough; users need to know what the system is doing, how long it has been running, and whether their progress is safe.

Limitations and Future Work

Current Limitations

- Free provider latency and availability are inconsistent, so the present local configuration is suitable for testing rather than a production service-level target.
- GitHub anonymous access is limited to 60 requests per hour. Repeated deep testing needs an authenticated token.
- The retrieval layer is lexical and has no semantic vector index.
- Scanned image-only PDFs are not processed because OCR is not implemented.
- Jira and Linear support is export-only; there is no ongoing synchronization.
- Sharing supports approval decisions but not simultaneous collaborative editing or comments.
- The evaluation dataset contains draft labels and still needs review with a mentor or domain expert.
- A full accessibility audit and physical-phone test pass are pending.

Recommended Next Steps

1. Add a read-only GitHub personal access token and record a repeatable retrieval baseline.
2. Use a stable paid or reserved-capacity model for production latency and availability tests.
3. Deploy a staging environment with Postgres, Upstash Redis, QStash, HTTPS, and at least one sign-in method.
4. Run the complete Playwright browser suite and test the application on a physical phone.
5. Review the 30 evaluation cases with the project guide, then run the real provider evaluation and record accuracy, latency, and cost.
6. Correct and verify the scheduled retention integration in staging before relying on automated cleanup.
7. Add OCR only if scanned client documents are a real project requirement.
8. Collect actual delivery data from at least five completed projects to validate the calibration loop.

Conclusion

Aureon demonstrates a complete workflow for turning uncertain requirements into a reviewable software scope. The project combines a modern full-stack application, structured model calls, public-code retrieval, deterministic scoring, background processing, secure storage, editable outputs, and a substantial automated test suite.

The implementation is functional for internal testing. The live pipeline completed successfully with the current OpenRouter routing, and the automated baseline now passes 1,035 tests. The remaining work is mainly operational: authenticated GitHub access, stable production model capacity, hosted infrastructure, browser and device testing, and a reviewed real-provider evaluation baseline.

From an engineering perspective, the project met its main objective. It does not treat generated text as the product by itself. It builds a controlled process around that text so users can understand what is known, what is missing, why a decision was made, and how the scope changes over time.

Appendix A Local Setup

The minimum local development environment requires Node.js 22 or newer, the project dependencies, and one working model provider key. PGlite, in-memory rate limiting, and local after() jobs are used automatically when hosted services are absent.

1. Install dependencies with npm install.
2. Copy .env.example to .env.local.
3. Configure LLM_PROVIDER and a valid server-side provider key.
4. Optionally add GITHUB_TOKEN for repeated GitHub enrichment tests.
5. Start the application with npm run dev -- --port 3210.
6. Open http://localhost:3210/chat and submit a detailed project brief.

Minimum Testing Commands

```
npm run typecheck
npm run lint
npm test
npm run build
$env:SCOPEAI_BASE_URL='http://localhost:3210'; npx tsx scripts/test-flow.ts
```

Appendix B Main Technologies

Technology	Use in the project
Next.js 16.3.8	Full-stack routing, server rendering, route handlers, and local background work
React 19.2.4	Interactive chat, editing, project views, and progress UI
TypeScript	Strict types across client, server, schemas, and tests
Tailwind CSS 4	Responsive layout and visual styling
GSAP and Framer Motion	Stage transitions and interface motion
Zod 4	Runtime validation of requests, model outputs, configuration, and stored data
Drizzle ORM	Typed Postgres schema and database access
PGlite	File-backed local Postgres fallback
Upstash Redis and QStash	Distributed coordination and durable production jobs
Octokit	GitHub repository search and content access
Vitest and Playwright	Unit, integration, and browser testing